

Algorytmy wyznaczania otoczki wypukłej (Convex Hull) - analiza porównawcza

Piotr Niepsuj

Spis treści

1. Wprowadzenie	2
1.1. Środowisko testowe i sprzęt	2
1.2. Metoda generowania danych testowych	2
1.3. Metoda testowania	3
2. Graham's scan	4
3. Andrew's Monotone Chain	7
4. Jarvis March	10
5. Quick Hull	14
6. Chan's Algorithm	18
7. Porównanie algorytmów	22
7.1. Punkty losowe	22
7.2. Punkty na obwodzie koła	23
7.3. Punkty na siatce	24
7.4. Punkty w skupisku	25
7.5. Punkty w trójkącie	26
8. Wnioski końcowe	27

1. Wprowadzenie

Celem tego projektu jest praktyczne zbadanie algorytmów wyznaczania wypukłej otoczki, a także porównanie długości czasu wykonania programu zależnie od wybranego algorytmu i scenariusza danych. Projekt zakłada sformułowanie własnych hipotez i ich empiryczną weryfikację.

W ramach projektu zaimplementowałem 5 popularnych algorytmów jakimi są Skanowanie Grahama, Łańcuch Monotoniczny, Marsz Jarvisa, QuickHull oraz algorytm Chan'a. Aby uzyskać pełny obraz tego, jak algorytmy zachowują się w różnych warunkach, każdy z nich jest testowany na pięciu scenariuszach danych wejściowych:

1. **Punkty losowe** - losowe punkty rozłożone równomiernie w kwadracie, gdzie $X, Y \in [-0.5, 0.5]$
2. **Punkty na obwodzie koła** - losowe punkty rozłożone równomiernie na obwodzie koła o $r = 1$, co jest szczególnie interesujące dla algorytmów wrażliwych na układ punktów
3. **Punkty na siatce** - losowe punkty rozmieszczone na regularnej siatce, co może być wyzwaniem dla algorytmów, które nie radzą sobie dobrze z dużą ilością współliniowych punktów
4. **Punkty w skupisku** - 90% punktów zostaje rozmieszczone w małym kole o $r = 1$, a pozostałe 10% w dużym kole o $r = 100$, co pozwala sprawdzić, jak algorytmy radzą sobie z danymi o bardzo zróżnicowanej gęstości
5. **Punkty w trójkącie** - losowe punkty rozmieszczone wewnątrz trójkąta ABC z tendencją do skupiania się przy krawędzi AB, co może sprawić algorytmom problemy podobne do układzie na siatce

1.1. Środowisko testowe i sprzęt

- **Procesor:** Intel Core i5-12600K
- **Pamięć RAM:** 32 GB (DDR4)
- **Język programowania:** C++
- **Kompilator:** g++ (GCC) 15.2.0

1.2. Metoda generowania danych testowych

Do przygotowania danych wejściowych stworzyłem program w języku C++. Program generuje zestawy danych dla wielkości bazujących na potęgach liczby 10. Aby zwiększyć rozdzielczość wykresów, potęgi te są dodatkowo zagęszczane przez mnożniki 1, 2 oraz 5 (co daje nam tablice o rozmiarach np. 10, 20, 50, 100, 200, 500, 1000 itd.). Wygenerowane dane zapisywane są do plików CSV, dzięki czemu każdy algorytm operuje na dokładnie takich samych liczbach w danym scenariuszu.

Program obsługuje się z wiersza poleceń, a zakres generowanych danych można łatwo dostosować (przedziały podawane są łącznie). Przykłady użycia:

- `./pnpr3 generate` - generuje domyślnie wszystkie dane wielkości od 1 do 4 potęgi 10.
- `./pnpr3 generate -s 2 -e 3` - ogranicza generowanie plików CSV tylko do rozmiarów od 2 do 3 potęgi 10.
- `./pnpr3 generate -p 1 -p 3 -p 5 -p 8` - zmiana mnożników z 1, 2, 5 na 1, 3, 5, 8.
- `./pnpr3 generate -d random -d circle` - generowanie danych tylko dla scenariuszy random i circle.

1.3. Metoda testowania

Ten sam program w C++ odpowiada za przeprowadzanie właściwych pomiarów. Aplikacja wczytuje przygotowane wcześniej dane z plików CSV, uruchamia wybrany algorytm i mierzy jego czas wykonania (z wykorzystaniem wbudowanych narzędzi biblioteki chrono).

Interfejs z poziomu konsoli pozwala na bardzo dużą elastyczność w dobieraniu testów, co jest szczególnie przydatne przy wolniejszych algorytmach, których nie chcemy puszczać dla ogromnych ilości punktów.

- `./pnpr3 test` - odpala wszystkie algorytmy na wszystkich 5 scenariuszach w przedziale od 1 do 4 potęg 10.
- `./pnpr3 test -s 2 -e 3` - testuje wszystkie algorytmy i scenariusze, ale zawęża zestaw danych do potęg od 2 do 3.
- `./pnpr3 test -p 1 -p 3 -p 5 -p 8` - zmiana mnożników z 1, 2, 5 na 1, 3, 5, 8.
- `./pnpr3 test -d random -d circle` - testuje wszystkie algorytmy tylko dla scenariuszy random i circle.
- `./pnpr3 test -a graham-scan -a monotone-chain` - testuje tylko algorytm graham-scan i monotone-chain.

Pojedynczy test na danym algorytmie, scenariuszu i rozmiarze danych generuje plik CSV zawierający próby od -10 do 99 i czas wykonania algorytmu w nanosekundach, przykład można zobaczyć na Tabeli 1. Przy wyliczaniu średniej do wykresów tylko próby od 0 do 99 są brane pod uwagę. Próby od -10 do -1 są na rozgrzewkę dla procesora.

Każdy test zostaje przeprowadzony na odizolowanym rdzeniu procesora z regulatorem skalowania ustawionym na wydajność. W moim przypadku korzystam z rdzenia 0, który jest rdzeniem wydajnościowym. Testy są także włączane z najwyższym możliwym priorytetem. Zostało to osiągnięte w następujący sposób

- **Izolacja rdzenia 0**
Dodanie do `/etc/default/grub` ustawienia
`GRUB_CMDLINE_LINUX_DEFAULT="isolcpus=0 nohz_full=0 rcu_nocbs=0"`
- **Ustawienie regulatora skalowania dla rdzenia 0 na wydajność (do restartu systemu)**
Wykonane za pomocą komendy `echo performance | sudo tee /sys/devices/system/cpu/cpu0/cpufreq/scaling_governor` można sprawdzić czy operacja powiodła się wypisując `cat /sys/devices/system/cpu/cpu0/cpufreq/scaling_governor`
- **Uruchomienie testów na rdzeniu 0 z największym priorytetem** Wykorzystanie `taskset` do przypisania procesu do rdzenia 0 oraz `chrt` do ustawienia najwyższego priorytetu. Komenda użyta w testach to `sudo chrt -f 99 taskset -c 0 ./main test`

Próba	Czas wykonania (ns)
-10	6456
-9	3573
-8	2853
-7	2642
-6	2637
-5	2261
-4	2209
-3	2110
-2	2088
-1	2093
0	2198
1	2105
2	2096
3	2137
4	2083
5	2077
6	2068
7	2072
...	...
99	2022

Tabela 1: Wyniki testowe z pliku CSV dla Skanowania Grahama na 100 punktach z losowym ułożeniem.

2. Graham's scan

Algorytm Skanowania Grahama to jedno z najbardziej klasycznych podejść do problemu wyznaczania otoczki wypukłej. Jego działanie opiera się na sprytnym posortowaniu punktów, a następnie odrzucaniu tych, które tworzą wklęsnięcia. Działanie algorytmu możemy podzielić na trzy etapy.

1. Wybór punktu startowego

Na początku musimy znaleźć punkt, który na pewno należy do otoczki wypukłej. Najprościej jest to zrobić wybierając najniższy punkt, czyli ten o najmniejszej współrzędnej Y . W przypadku kilku takich punktów, wybieramy ten, który leży najbardziej po lewej stronie, czyli ma najmniejszy X . Punkt ten nazwiemy P_0 - będzie on naszym punktem odniesienia do końca algorytmu.

2. Sortowanie kątowe

Teraz bierzemy wszystkie pozostałe punkty i sortujemy je rosnąco względem kąta, jaki tworzą z punktem P_0 oraz osią X . Wykorzystujemy do tego znak iloczynu wektorowego (cross product), jeśli podczas sortowania okaże się, że kilka punktów leży na jednej prostej z punktem P_0 , zostawiamy tylko ten, który jest od niego najdalej, ponieważ pozostałe i tak leżałyby wewnątrz krawędzi otoczki, więc od razu je odrzucamy, co oszczędza niepotrzebnych obliczeń.

3. Skanowanie przy użyciu stosu

Towrzymy stos i wrzucamy na niego nasz punkt startowy P_0 oraz dwa pierwsze punkty z posortowanej listy, następnie iterujemy przez resztę posortowanych punktów. Dla każdego nowego punktu sprawdzamy ułożenie przedostatniego punktu na stosie, ostatniego punktu na stosie (na samym szczycie) i nowego punktu, który właśnie rozpatrujemy. Chcemy, aby te trzy punkty zawsze tworzyły skręt w lewo. Jeśli sprawdzany nowy punkt sprawia, że tworzy się skręt w prawo, oznacza to, że punkt znajdujący się obecnie na szczycie stosu tworzy wklęsnięcie i nie może należeć do otoczki wypukłej. Usuwamy ten punkt ze stosu i sprawdzamy skręt ponownie dla poprzednich punktów. Zdejmujemy punkty tak długo, aż znów uzyskamy poprawny skręt w lewo. Gdy to nastąpi, dodajemy nasz nowy punkt na stos i przechodzimy do kolejnego.

Po przejściu przez całą posortowaną listę, na stosie zostają nam wyłącznie wierzchołki naszej otoczki wypukłej uporządkowane w kierunku przeciwnym do ruchu wskazówek zegara.

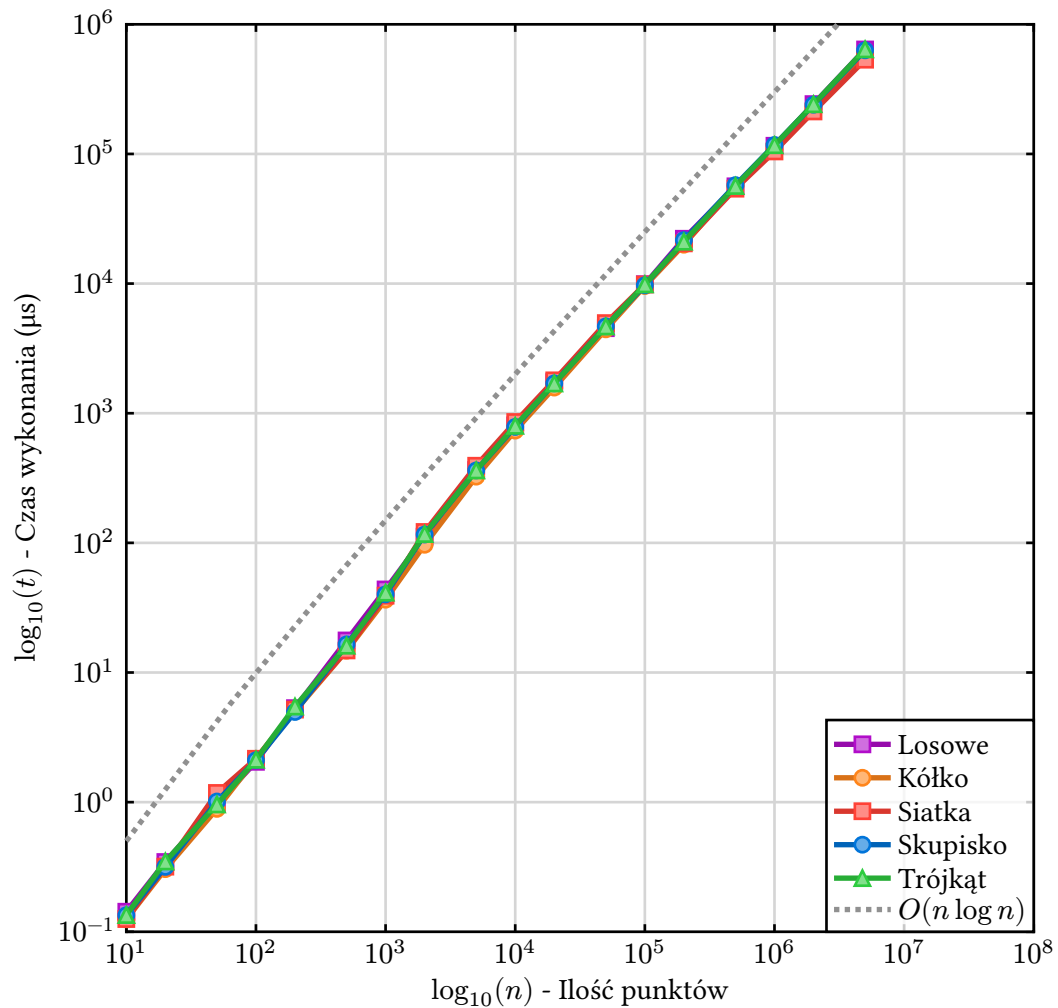
Algorytm 1: Skanowanie Grahama

```
1: procedure GRAHAMSCAN(points)
2:    $n \leftarrow$  ilość points
3:   if  $n < 3$  then
4:     return points
5:   end
6:
7:   ▷ Znajdź najniższy punkt (w przypadku remisu wysunięty najbardziej w lewo)
8:    $\text{min}_{\text{idx}} \leftarrow 0$ 
9:    $i \leftarrow 1$ 
10:  while  $i < n$  do
11:    if  $\text{points}[i].y < \text{points}[\text{min}_{\text{idx}}].y - \varepsilon$  lub  $(|\text{points}[i].y - \text{points}[\text{min}_{\text{idx}}].y| < \varepsilon$  i
12:       $\text{points}[i].x < \text{points}[\text{min}_{\text{idx}}].x - \varepsilon)$  then
13:       $\text{min}_{\text{idx}} \leftarrow i$ 
14:    end
15:     $i \leftarrow i + 1$ 
16:  end
17:  zamień  $\text{points}[0]$  z  $\text{points}[\text{min}_{\text{idx}}]$ 
18:   $P_0 \leftarrow \text{points}[0]$ 
19:  ▷ Sortowanie kątowe i filtrowanie
20:  posortuj  $\text{points}[1..n - 1]$  rosnąco względem kąta tworzonego z wektorem od  $P_0$ 
21:  jeśli kąty są równe, bliższy punkt umieść przed dalszym
22:  odfiltruj współliniowe punkty w points, zostawiając tylko najdalsze od  $P_0$  na danej
    półprostej
23:   $m \leftarrow$  ilość points po odfiltrowaniu
24:  if  $m < 3$  then
25:    return  $\text{points}[0..m - 1]$ 
26:  end
27:
28:  ▷ Budowa otoczki ze stosem
29:  hull  $\leftarrow$  pusty stos
30:  dodaj  $\text{points}[0]$ ,  $\text{points}[1]$ ,  $\text{points}[2]$  na stos hull
31:
32:   $i \leftarrow 3$ 
33:  while  $i < m$  do
34:    while  $\text{rozmiar}(\text{hull}) > 1$  do
35:      top  $\leftarrow$  szczyt stosu hull
36:      next_to_top  $\leftarrow$  element tuż pod szczytem stosu hull
37:
38:      if  $\text{next\_to\_top.cross\_product}(\text{top}, \text{points}[i]) < \varepsilon$  then
39:        zdejmij element ze stosu hull
40:      else
41:        BREAK
42:      end
43:    end
44:    dodaj  $\text{points}[i]$  na stos hull
45:     $i \leftarrow i + 1$ 
46:  end
47:
48:  return hull
49: end
```

Hipotezy badawcze

Przewiduję bardzo zbliżony czas wykonania niezależnie od początkowego układu danych. Na wykresie linie dla poszczególnych przypadków prawdopodobnie będą się na siebie nakładać w okolicach złożoności $O(n \log n)$. Wynika to z faktu, że algorytm nie bierze pod uwagę ułożenia

punktów w celu optymalizacji – i tak zawsze musi posortować całą pulę punktów, a potem przejść przez każdy z nich na stosie, nie pomijając żadnego kroku.



Rysunek 1: Czas wykonania Graham's scan - porównanie scenariuszy danych wejściowych z linią referencyjną $O(n \log n)$

Analizując Rysunek 1, widać wyraźnie, że linie dla wszystkich badanych scenariuszy nakładają się na siebie. Zgodnie z moją hipotezą, początkowy układ punktów nie ma żadnego wpływu na czas wykonania programu. Wykres idealnie podąża za złożonością $O(n \log n)$. Dzieje się tak, ponieważ algorytm w każdym przypadku musi najpierw posortować wszystkie wejściowe punkty, co w pełni dominuje całkowity czas obliczeń. Następnie program przechodzi przez posortowaną listę i na bieżąco aktualizuje stos punktów tworzących otoczkę, co zajmuje czas liniowy. Ponieważ krok sortowania jest wymagany zawsze, niezależnie od tego, czy punkty leżą na obwodzie koła, na siatce czy tworzą gęste skupisko, czas wykonania pozostaje taki sam dla każdego możliwego układu danych.

Czasowa złożoność obliczeniowa

- **Najgorszy, średni i najlepszy przypadek:** $O(n \log n)$ - Wynika to z faktu, że pierwszym krokiem algorytmu jest zawsze posortowanie wszystkich wejściowych punktów, co zajmuje czas $O(n \log n)$. Ta faza jest obowiązkowa i bezwzględnie dominuje nad drugą fazą algorytmu (przeszukiwaniem ze stosu w czasie $O(n)$). Z tego powodu Graham's scan nie jest wrażliwy na to, jak ułożone są punkty ani jak wiele z nich ostatecznie znajdzie się na powłoce wypukłej.

3. Andrew's Monotone Chain

Algorytm Łańcucha Monotonicznego to modyfikacja Skanowania Grahama. Główne ulepszenie polega na zastąpieniu sortowania kąтового sortowaniem leksykograficznym, co jest zazwyczaj szybsze i mniej podatne na błędy precyzji arytmetyki zmiennoprzecinkowej. Działanie algorytmu możemy podzielić na trzy etapy.

1. Sortowanie leksykograficzne

Na początku sortujemy wszystkie punkty rosnąco według współrzędnej X . Jeśli kilka punktów ma taką samą współrzędną X , sortujemy je rosnąco według współrzędnej Y . W ten sposób ustawiamy punkty w kolejności od najbardziej wysuniętego na lewo do najbardziej wysuniętego na prawo. Ewentualne duplikaty punktów na tym etapie można od razu usunąć.

2. Budowa dolnej otoczki (Lower Hull)

Tworzymy pustą listę działającą jak stos, na której będziemy budować dolną otoczkę. Iterujemy przez posortowane punkty od pierwszego do ostatniego. Podobnie jak w Skanowaniu Grahama, dla każdego dodawanego punktu analizujemy przedostatni punkt na stosie, szczyt stosu oraz rozpatrywany punkt. Sprawdzamy, czy tworzą one skręt w lewo, wykorzystując do tego znak iloczynu wektorowego. Jeśli tworzą skręt w prawo lub linię prostą, zdejmujemy element ze szczytu stosu tak długo, aż uzyskamy skręt w lewo, a następnie dodajemy nowy punkt. Po przejściu całej listy uzyskujemy dolną połowę otoczki.

3. Budowa górnej otoczki (Upper Hull)

Tym razem iterujemy przez posortowaną listę punktów w odwrotnej kolejności - od przedostatniego do pierwszego punktu. Nowe punkty dodajemy bezpośrednio na ten sam stos, cały czas weryfikując warunek skrętu w lewo. Pozwala nam to zamknąć wielokąt, wracając do punktu startowego. Na samym końcu usuwamy ostatni dodany element, ponieważ w wyniku iteracji punkt startowy został dodany dwukrotnie. Otrzymujemy pełną otoczkę wypukłą uporządkowaną w kierunku przeciwnym do ruchu wskazówek zegara.

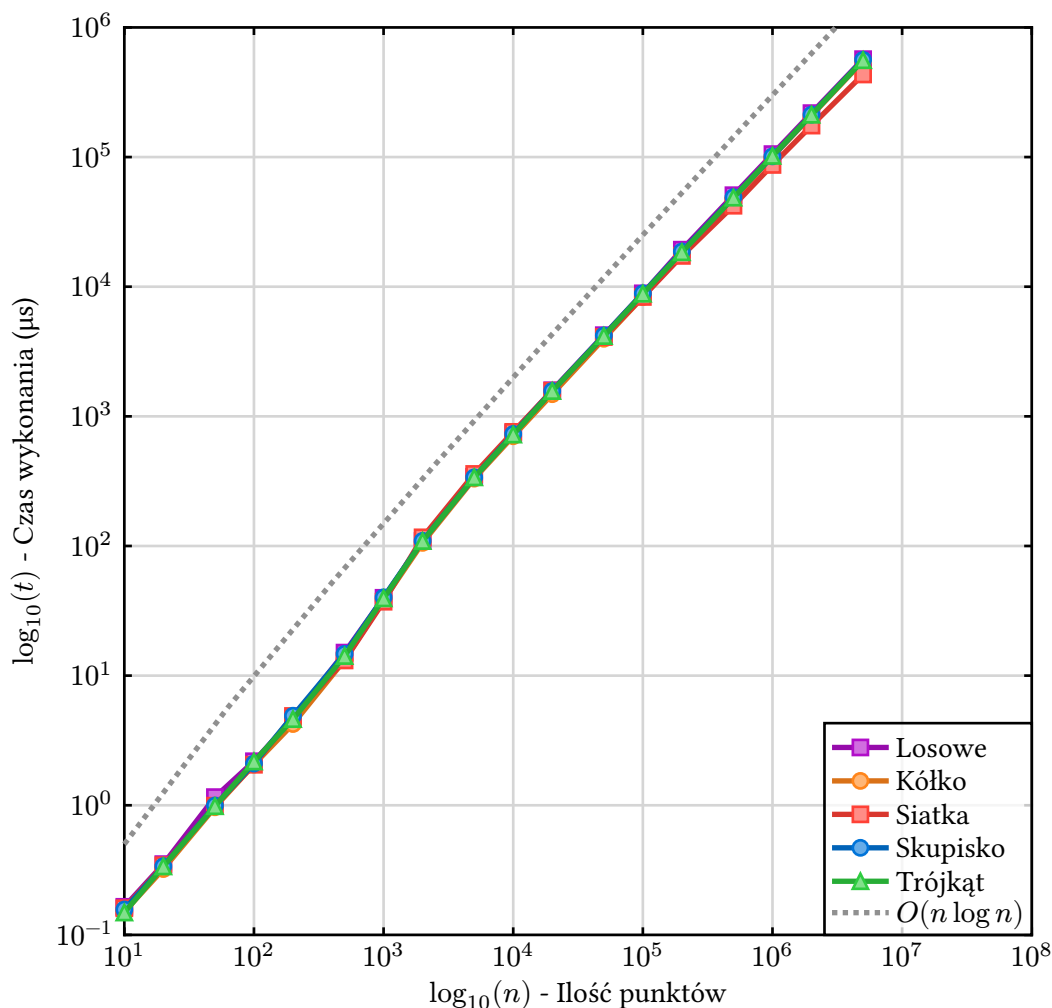
Algorytm 2: Łańcuch Monotoniczny

```
1: procedure MONOTONECHAIN(points)
2:    $n \leftarrow$  ilość points
3:   if  $n < 3$  then
4:     return points
5:   end
6:
7:   ▷ Sortowanie leksykograficzne
8:   posortuj points rosnąco według  $X$ , a w przypadku remisu według  $Y$ 
9:
10:  hull  $\leftarrow$  pusty stos
11:
12:  ▷ Budowa dolnej otoczki
13:   $i \leftarrow 0$ 
14:  while  $i < n$  do
15:    while rozmiar(hull)  $\geq 2$  do
16:      top  $\leftarrow$  szczyt stosu hull
17:      next_to_top  $\leftarrow$  element tuż pod szczytem stosu hull
18:
19:      if next_to_top.cross_product(top, points[ $i$ ])  $< \varepsilon$  then
20:        zdejmij element ze stosu hull
21:      else
22:        BREAK
23:      end
24:    end
25:    dodaj points[ $i$ ] na stos hull
26:     $i \leftarrow i + 1$ 
27:  end
28:
29:  ▷ Budowa górnej otoczki
30:   $t \leftarrow$  rozmiar(hull) + 1
31:   $i \leftarrow n - 2$ 
32:  while  $i \geq 0$  do
33:    while rozmiar(hull)  $\geq t$  do
34:      top  $\leftarrow$  szczyt stosu hull
35:      next_to_top  $\leftarrow$  element tuż pod szczytem stosu hull
36:
37:      if next_to_top.cross_product(top, points[ $i$ ])  $< \varepsilon$  then
38:        zdejmij element ze stosu hull
39:      else
40:        BREAK
41:      end
42:    end
43:    dodaj points[ $i$ ] na stos hull
44:     $i \leftarrow i - 1$ 
45:  end
46:
47:  ▷ Ostatni dodany punkt to punkt startowy dolnej otoczki - usuwamy duplikat
48:  zdejmij element ze stosu hull
49:
50:  return hull
51: end
```

Hipotezy badawcze

Podobnie jak w przypadku algorytmu Grahama, przewiduję, że wykresy dla wszystkich zestawów danych nałożą się na siebie, tworząc linię o rzędzie $O(n \log n)$. Główny czas wykonania jest zdominowany przez początkowe sortowanie leksykograficzne (po współrzędnych X i Y). Sama pętla

budująca górną i dolną część otoczki zawsze przechodzi przez wszystkie n punktów z listy, więc ich początkowy układ przestrzenny nie powinien wygenerować zauważalnych różnic czasowych na wykresie.



Rysunek 2: Czas wykonania Andrew's Monotone Chain - porównanie scenariuszy danych wejściowych z linią referencyjną $O(n \log n)$

Analizując Rysunek 2, widać wyraźnie, że linie dla wszystkich badanych scenariuszy nakładają się na siebie. Zgodnie z moją hipotezą, początkowy układ punktów nie ma wpływu na czas wykonania programu. Wykres podąża za złożonością $O(n \log n)$. Dzieje się tak z tego samego powodu co przy Skanowaniu Grahama - algorytm w każdym przypadku musi najpierw posortować wszystkie wejściowe punkty leksykograficznie po współrzędnych, co dominuje całkowity czas obliczeń. Następnie program przechodzi przez posortowaną listę, budując górną i dolną część otoczki, co zajmuje czas liniowy. Ponieważ krok sortowania jest wymagany zawsze, czas wykonania pozostaje taki sam dla każdego możliwego układu danych.

Czasowa złożoność obliczeniowa

- **Najgorszy, średni i najlepszy przypadek:** $O(n \log n)$ - Wynika to z faktu, że pierwszym krokiem algorytmu jest zawsze posortowanie wszystkich wejściowych punktów, co zajmuje czas $O(n \log n)$. Z tego powodu algorytm - tak jak Skanowanie Grahama - nie jest wrażliwy na ułożenie przestrzenne punktów ani na to, ile z nich ostatecznie znajdzie się na powłoce wypukłej.

4. Jarvis March

Algorytm Marszu Jarvisa, znany również jako owijanie prezentu (Gift Wrapping), to algorytm bardzo wrażliwy na rozmiar wyjścia, czyli to, ile punktów znajdzie się na otoczce wypukłej. Jego działanie możemy podzielić na dwa etapy.

1. Znalezienie punktu startowego

Podobnie jak w innych algorytmach, na początku musimy znaleźć punkt, który z całą pewnością znajduje się na krawędzi otoczki wypukłej. Szukamy punktu najbardziej wysuniętego w lewo, czyli tego o najmniejszej współrzędnej X . Jeśli kilka punktów ma tak samo małą współrzędną X , wybieramy z nich ten o najmniejszej współrzędnej Y .

2. Owijanie prezentu

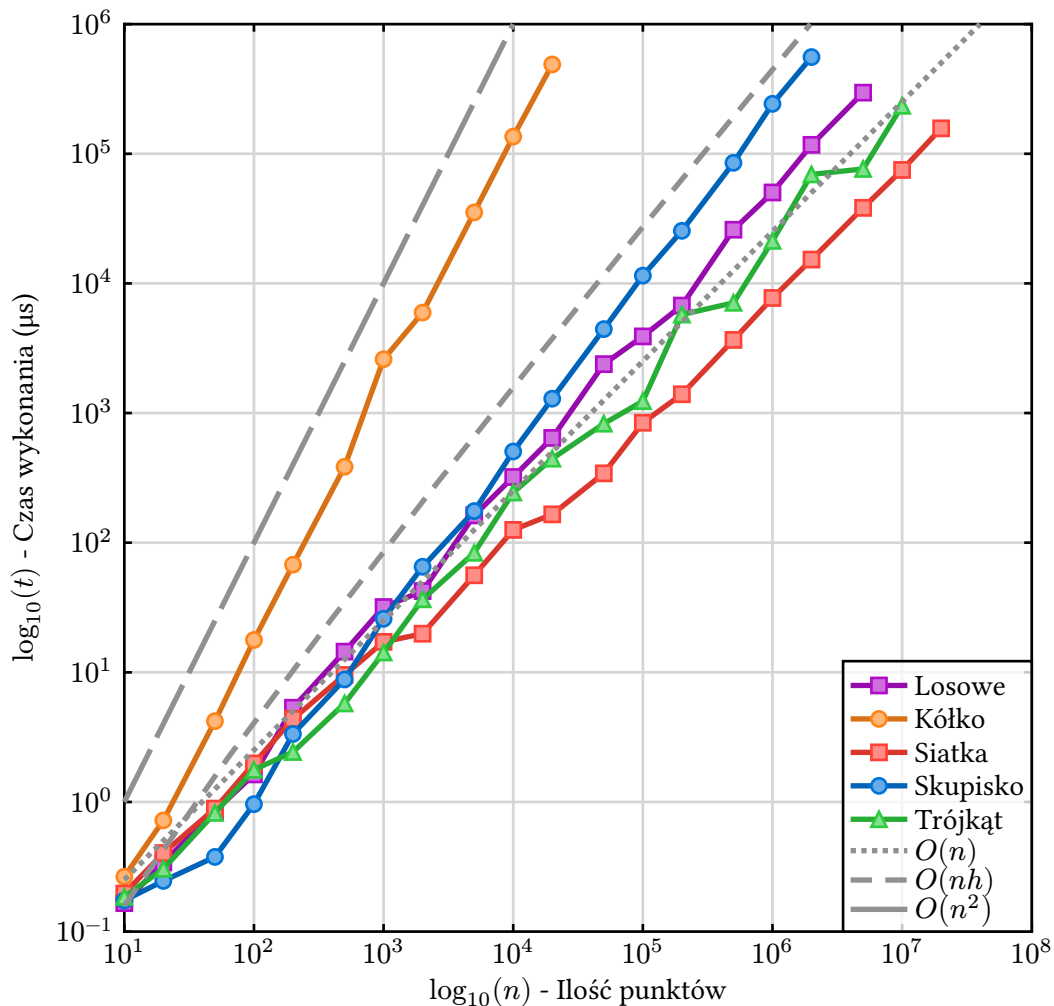
Wybrany punkt startowy dodajemy do naszej otoczki i oznaczamy go jako nasz punkt bieżący. Następnie rozpoczynamy proces poszukiwania kolejnego wierzchołka, który przypomina owijanie zbioru punktów napiętym sznurkiem. Aby to zrobić, na początku iteracji wybieramy dowolny inny punkt ze zbioru i oznaczamy go jako naszego tymczasowego kandydata. Następnie sprawdzamy po kolei wszystkie pozostałe punkty. Dla każdego punktu badamy znak iloczynu wektorowego, aby upewnić się, czy leży on po lewej stronie wektora łączącego punkt bieżący z naszym kandydatem. Jeśli tworzy skręt w lewo, oznacza to, że leży bardziej na zewnątrz, więc staje się naszym nowym kandydatem. Jeśli analizowany punkt jest współliniowy z obecnym kandydatem, wykorzystujemy iloczyn skalarny, aby upewnić się, że oba punkty leżą w tym samym kierunku. Jeśli tak jest, wybieramy ten, który jest dalej od punktu bieżącego. Po sprawdzeniu całej listy punktów ostateczny zwycięzca zostaje dodany jako nowy wierzchołek otoczki wypukłej. Staje się on naszym nowym punktem bieżącym, a całą procedurę powtarzamy aż do momentu, w którym nowo wyłoniony wierzchołek okaże się naszym punktem startowym, co oznacza pomyślne zamknięcie obwodu.

Algorytm 3: Marsz Jarvisa

```
1: procedure JARVISMARCH(points)
2:    $n \leftarrow$  ilość points
3:   if  $n < 3$  then
4:     return points
5:   end
6:
7:   hull  $\leftarrow$  pusty stos
8:
9:    $\triangleright$  Znajdź punkt startowy (najbardziej wysunięty w lewo)
10:  leftmost  $\leftarrow 0$ 
11:   $i \leftarrow 1$ 
12:  while  $i < n$  do
13:    if  $\text{points}[i].x < \text{points}[\text{leftmost}].x - \varepsilon$  lub  $(|\text{points}[i].x - \text{points}[\text{leftmost}].x| < \varepsilon \wedge$ 
14:       $\text{points}[i].y < \text{points}[\text{leftmost}].y - \varepsilon)$  then
15:      leftmost  $\leftarrow i$ 
16:    end
17:     $i \leftarrow i + 1$ 
18:  end
19:   $p \leftarrow \text{leftmost}$ 
20:
21:   $\triangleright$  Owijanie prezentu
22:  while prawda do
23:    dodaj  $\text{points}[p]$  na stos hull
24:
25:     $q \leftarrow (p + 1) \bmod n$ 
26:    while  $\text{points}[p].\text{dist\_sq}(\text{points}[q]) < \varepsilon \wedge q \neq p$  do
27:       $q \leftarrow (q + 1) \bmod n$ 
28:    end
29:
30:     $i \leftarrow 0$ 
31:    while  $i < n$  do
32:      if  $i \neq p \wedge i \neq q$  then
33:         $\text{cp} \leftarrow \text{points}[p].\text{cross\_product}(\text{points}[i], \text{points}[q])$ 
34:
35:        if  $\text{cp} > \varepsilon$  then
36:           $q \leftarrow i$ 
37:        else
38:          if  $|\text{cp}| \leq \varepsilon$  then
39:            dot  $\leftarrow$  iloczyn skalarny wektorów od  $p$  do  $i$  oraz od  $p$  do  $q$ 
40:            if dot  $> 0$  i  $\text{points}[p].\text{dist\_sq}(\text{points}[i]) > \text{points}[p].\text{dist\_sq}(\text{points}[q])$  then
41:               $q \leftarrow i$ 
42:            end
43:          end
44:        end
45:         $i \leftarrow i + 1$ 
46:      end
47:
48:       $p \leftarrow q$ 
49:      if  $p = \text{leftmost}$  then
50:        BREAK
51:      end
52:    end
53:
54:    return hull
55: end
```

Hipotezy badawcze dla poszczególnych scenariuszy

1. **Punkty losowe:** Liczba punktów tworzących faktyczną otoczkę powinna być ułamkiem wszystkich punktów, więc czas wykonania będzie umiarkowany.
2. **Punkty na obwodzie koła:** worst-case - Przewiduję, że ten wykres wystrzeli najwyżej ze wszystkich. Ponieważ każdy punkt leży na obwodzie, algorytm najpewniej weźmie każdy z nich pod uwagę jako wierzchołek otoczki, robiąc pełne przejście dla każdego punktu. Spodziewam się krzywej przypominającej $O(n^2)$.
3. **Punkty na siatce:** best-case – Przy rosnącej liczbie punktów siatka wypełnia się równomiernie, a otoczka zawiera jedynie punkty narożne i brzegowe. Wraz ze wzrostem n wartość h zmierza do stałej, więc spodziewam się wykresu niemal liniowego, najlepszego spośród wszystkich scenariuszy.
4. **Punkty w skupisku:** Oczekuję, że wykres wylądjuje bardzo nisko, blisko best-case'u. Algorytm najpewniej oprze się tylko na kilku punktach z rzadkiego, zewnętrznego koła, omijając żmudną analizę gęstego środka jako potencjalnych wierzchołków.
5. **Punkty w trójkącie:** Punkty skupiają się blisko krawędzi AB , a otoczka jest przybliżeniem trójkąta ABC – jej wierzchołki to skrajne punkty zbioru, nie dokładnie A, B, C . Wartość h pozostaje mała i w przybliżeniu stała, więc oczekuję czasu zbliżonego do scenariusza z siatką, choć nieco gorszego ze względu na skupisko przy AB mogące generować kilka dodatkowych punktów na otoczce.



Rysunek 3: Czas wykonania Jarvis March - porównanie scenariuszy danych wejściowych z liniami referencyjnymi $O(n)$, $O(nh)$ i $O(n^2)$, gdzie $h = \sqrt{n}$

Analizując Rysunek 3, wyraźnie widać, jak silnie czas jego wykonania zależy od liczby punktów na otoczce wypukłej, co w pełni ukazuje jego wrażliwość na ten parametr (output-sensitivity). Zgodnie z moją hipotezą, najgorzej wypadły punkty na obwodzie koła. Wykres dla tego przypadku pnie się w górę ze złożonością $O(n^2)$. Dzieje się tak, ponieważ każdy punkt z tego zbioru ostatecznie trafia na otoczkę, zmuszając algorytm do zrobienia pełnego przejścia po wszystkich punktach w każdym z n kroków. Jako najlepszy przypadek, dokładnie tak jak zakładałem, sprawdziły się punkty na siatce. Otoczkę tworzy tu tylko garstka punktów skrajnych, przez co czas działania rośnie bardzo wolno i zbliża się do funkcji liniowej $O(n)$. Podobnie dobrze wypadł scenariusz z trójkątem, gdzie wykres utrzymuje się nisko ze względu na bardzo małą wartość h , co również pokrywa się z moimi przewidywaniami. Dane losowe zachowały się umiarkowanie, ukazując się pomiędzy wszystkimi scenariuszami i podążając wzdłuż linii $O(nh)$. Hipotezę dotyczącą punktów w skupisku nie okazała się trafna, oczekiwałem, że wykres znajdzie się bardzo nisko, blisko najlepszego przypadku. Tymczasem dane z wykresu pokazują, że scenariusz ten wypada znacznie gorzej od zwykłych danych losowych. Wynika to najprawdopodobniej z faktu, że aż 10% punktów zostało rozrzuconych na bardzo dużym, zewnętrznym okręgu, co sprawiło, że parametr h (liczba punktów na otoczce) stał się relatywnie duży. Ponieważ Jarvis March w każdym kroku budowania otoczki musi iterować po wszystkich n punktach, duża wartość h wywindowała ostateczny czas obliczeń.

Czasowa złożoność obliczeniowa

- **Najgorszy przypadek:** $O(n^2)$ - Wynika to z sytuacji, w której wszystkie podane punkty leżą na powłoce wypukłej. Algorytm wykonuje wtedy n kroków, a w każdym z nich musi przeszukać pozostałe $n - 1$ punktów, aby znaleźć kolejny wierzchołek, co daje złożoność kwadratową.
- **Średni przypadek:** $O(nh)$ - Czas działania algorytmu zależy liniowo zarówno od całkowitej liczby punktów n , jak i od liczby punktów tworzących samą otoczkę h . Dla typowych, losowych rozkładów jest to czas pośredni między liniowym a kwadratowym.
- **Najlepszy przypadek:** $O(n)$ - Występuje w sytuacji, gdy liczba wierzchołków na otoczce jest stała i bardzo mała. Przykładowo, dla punktów ułożonych na regularnej siatce, otoczkę tworzą zaledwie cztery rogi, dzięki czemu algorytm wykonuje tylko kilka pełnych iteracji i kończy pracę w czasie zbliżonym do liniowego.

5. Quick Hull

Algorytm QuickHull to metoda oparta na strategii dziel i rządź, przypominająca w swoim zachowaniu algorytm sortowania QuickSort. Działanie algorytmu możemy podzielić na trzy etapy.

1. Znalezienie punktów skrajnych

Na początku algorytm przeszukuje zbiór w celu znalezienia dwóch punktów skrajnych - najbardziej wysuniętego na lewo (oznaczymy go jako A) oraz najbardziej wysuniętego na prawo (oznaczymy go jako B). W przypadku remisów na osi X , dla punktu A wybieramy ten najniższy, a dla punktu B ten najwyższy. Punkty te z całą pewnością należą do otoczki wypukłej. Prosta przechodząca przez punkty A i B dzieli pozostały zbiór punktów na dwie podgrupy - punkty znajdujące się po lewej stronie wektora skierowanego od A do B oraz te po lewej stronie wektora od B do A .

2. Szukanie najdalszego punktu

Dla rozpatrywanej podgrupy punktów i tworzącego ją wektora, szukamy punktu, który znajduje się ściśle po jego lewej stronie i jest od niego najbardziej oddalony. Do sprawdzenia strony oraz proporcji odległości wykorzystujemy iloczyn wektorowy. Jeśli kilka punktów znajduje się w tej samej, maksymalnej odległości, wybieramy ten, który jest najdalej od punktu początkowego A . Ten nowo znaleziony, najdalszy punkt (nazwijmy go C) musi być wierzchołkiem otoczki. Co ważne, wszystkie punkty znajdujące się wewnątrz trójkąta ABC nie mogą należeć do krawędzi otoczki, dlatego algorytm w naturalny sposób przestanie je brać pod uwagę.

3. Wywołania rekurencyjne

Po znalezieniu punktu C , obszar poszukiwań zostaje podzielony na dwa mniejsze. Algorytm wywołuje sam siebie dla dwóch nowych wektorów - od A do C oraz od C do B . Szukamy teraz punktów znajdujących się po lewej stronie tych nowych krawędzi. Jeśli podczas wywołania okaże się, że po lewej stronie badanego wektora nie ma już żadnych punktów, oznacza to, że jego punkt końcowy jest docelowym wierzchołkiem naszej otoczki i dodajemy go do stosu wynikowego. Odpowiednia kolejność wywołań rekurencyjnych gwarantuje, że otrzymane wierzchołki będą poprawnie uporządkowane w kierunku przeciwnym do ruchu wskazówek zegara. Po zakończeniu wszystkich powrotów z rekurencji, usuwamy ewentualny duplikat punktu startowego na końcu listy.

Algorytm 4: QuickHull

```
1: procedure FINDHULL(points, A, B, hull)
2:   farthest_idx  $\leftarrow$  -1
3:   max_dist  $\leftarrow$  -1
4:    $n \leftarrow$  ilość points
5:    $i \leftarrow$  0
6:
7:   while  $i < n$  do
8:     cp  $\leftarrow$  A.cross_product(B, points[i])
9:     if cp  $> \epsilon$  then
10:      dist  $\leftarrow$  odległość points[i] od prostej wyznaczonej przez A i B
11:      if dist  $> \max_{\text{dist}} + \epsilon$  then
12:        max_dist  $\leftarrow$  dist
13:        farthest_idx  $\leftarrow$  i
14:      else
15:        if  $|\text{dist} - \max_{\text{dist}}| < \epsilon$  then
16:          if farthest_idx  $\neq$  -1 i A.dist_sq(points[i])  $>$  A.dist_sq(points[farthest_idx]) then
17:            farthest_idx  $\leftarrow$  i
18:          end
19:        end
20:      end
```

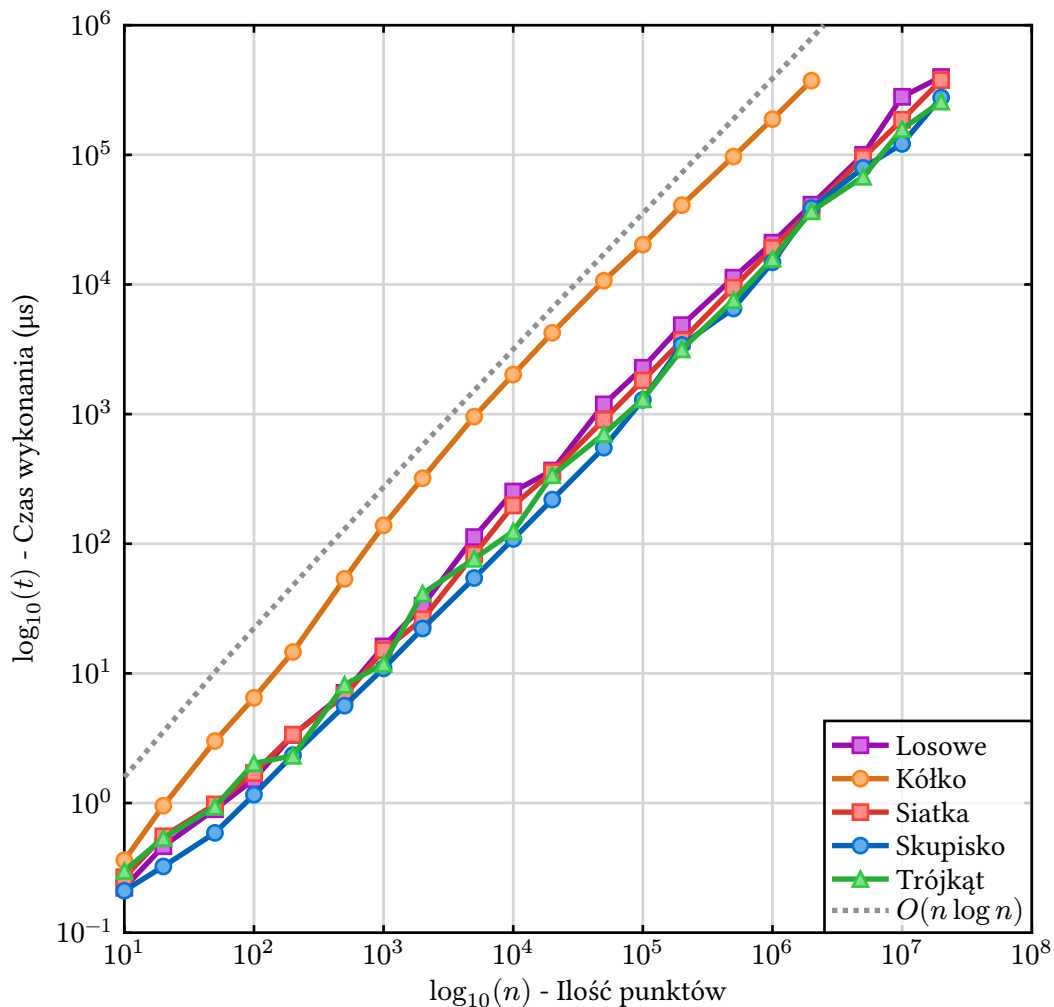
```

21:   end
22:    $i \leftarrow i + 1$ 
23: end
24:
25: if farthestidx = -1 then
26:   dodaj  $B$  na stos hull
27:   return
28: end
29:
30:  $C \leftarrow \text{points}[\text{farthest}_{\text{idx}}]$ 
31:
32: ▷ Przekazuj dalej tylko punkty znajdujące się ściśle po lewej stronie nowych krawędzi
33: leftAC ← pusta lista
34: leftCB ← pusta lista
35: for każdego punktu  $p$  w points do
36:   if  $A.\text{cross\_product}(C, p) > \varepsilon$  then
37:     dodaj  $p$  do leftAC
38:   end
39:   if  $C.\text{cross\_product}(B, p) > \varepsilon$  then
40:     dodaj  $p$  do leftCB
41:   end
42: end
43:
44: FINDHULL(leftAC,  $A$ ,  $C$ , hull)
45: FINDHULL(leftCB,  $C$ ,  $B$ , hull)
46: end
47:
48: procedure QUICKHULL(points)
49:   ▷ Znajdź punkty skrajne  $A$  (najbardziej w lewo) i  $B$  (najbardziej w prawo)
50:    $n \leftarrow$  ilość points
51:   minx ← 0
52:   maxx ← 0
53:    $i \leftarrow 1$ 
54:   while  $i < n$  do
55:     if points[ $i$ ]. $x < \text{points}[\text{min}_x].x - \varepsilon$  lub ( $|\text{points}[i].x - \text{points}[\text{min}_x].x| < \varepsilon$  i
56:       points[ $i$ ]. $y < \text{points}[\text{min}_x].y - \varepsilon$ ) then
57:       minx ←  $i$ 
58:     end
59:     if points[ $i$ ]. $x > \text{points}[\text{max}_x].x + \varepsilon$  lub ( $|\text{points}[i].x - \text{points}[\text{max}_x].x| < \varepsilon$  i
60:       points[ $i$ ]. $y > \text{points}[\text{max}_x].y + \varepsilon$ ) then
61:       maxx ←  $i$ 
62:     end
63:      $i \leftarrow i + 1$ 
64:   end
65:
66:    $A \leftarrow \text{points}[\text{min}_x]$ 
67:    $B \leftarrow \text{points}[\text{max}_x]$ 
68:   hull ← pusty stos
69:   dodaj  $A$  na stos hull
70:
71:   FINDHULL(points,  $A$ ,  $B$ , hull)
72:   FINDHULL(points,  $B$ ,  $A$ , hull)
73:
74:   if rozmiar(hull) > 1 i hull[0] = hull[ostatni element] then
75:     zdejmij element ze stosu hull
76:   end
77:   return hull
78: end

```

Hipotezy badawcze dla poszczególnych scenariuszy

1. **Punkty losowe:** Oczekuję czasu zbliżonego do $O(n \log n)$. Podział na pół-płaszczyzny skutecznie eliminuje dużą część punktów wewnętrznych, więc algorytm powinien radzić sobie dobrze.
2. **Punkty na obwodzie koła:** worst-case – Każdy punkt trafia na otoczkę, rekurencyjny podział nie eliminuje żadnych punktów, co powinno prowadzić do wykresu bliskiego $O(n^2)$. Spodziewam się najdłuższego czasu wykonania.
3. **Punkty na siatce:** best-case – Podobnie jak w Jarvis March, h zmierza do stałej wraz ze wzrostem n . Podział na pół-płaszczyzny eliminuje niemal wszystkie punkty już na pierwszych poziomach rekurencji, spodziewam się wykresu niemal liniowego.
4. **Punkty w skupisku:** Zewnętrzne koło wyznacza otoczkę, a gęste skupisko wewnętrzne zostaje szybko odrzucone przez podział. Oczekuję czasu zbliżonego do danych losowych.
5. **Punkty w trójkącie:** Skupisko przy krawędzi AB może sprawić, że kolejne poziomy rekurencji będą musiały przetwarzać wiele punktów leżących blisko otoczki. Oczekuję czasu nieco gorszego niż dla danych losowych, ale nadal wyraźnie lepszego niż dla okręgu.



Rysunek 4: Czas wykonania QuickHull - porównanie scenariuszy danych wejściowych z linią referencyjną $O(n \log n)$

Analizując Rysunek 4, od razu rzuca się w oczy potężna różnica między punktami na obwodzie koła a całą resztą badanych scenariuszy. Zgodnie z hipotezą, układ na kole okazał się najgorszym możliwym przypadkiem, jednak okazuje się podążać złożoność $O(n \log n)$, a nie $O(n^2)$ tak jak bym się tego spodziewał. Scenariusz z punktami na obwodzie koła wydaje się po prostu dodawać spory overhead przy aktualnej implementacji, co jest lepsze niż całkowita degradacja do czasu

kwadratowego. Pozostałe scenariusze, czyli dane losowe, siatka, skupisko i trójkąt, utworzyły jedną, bardzo zbitą wiązkę reprezentującą czas $O(n \log n)$. Spodziewałem się, że punkty na siatce okażą się najlepszym przypadkiem o czasie niemal liniowym, tymczasem wykres pokazuje, że algorytm przetwarza je w takim samym tempie jak zwykle dane losowe. Wynika to z faktu, że QuickHull i tak musi systematycznie dzielić płaszczyznę i przeszukiwać podzbiory rekurencyjnie, nawet przy małej liczbie punktów na powłoce. Ponadto, zakładałem, że zbiór w trójkącie sprawi lekkie problemy przez zagęszczenie przy jednej z krawędzi, ale w rzeczywistości absolutnie nie spowolniło to działania programu. Algorytm poradził sobie z tym zadaniem, podobnie jak z gęstym skupiskiem wewnętrznym, równie sprawnie co z danymi w pełni losowymi, potwierdzając swoją wysoką skuteczność w masowym odrzucaniu punktów nienależących do powłoki już na wczesnych etapach.

Czasowa złożoność obliczeniowa

- **Najgorszy przypadek:** $O(n^2)$ - Zgodnie z teorią występuje w sytuacji, gdy podział na pół-płaszczyzny nie eliminuje żadnych punktów (np. wszystkie leżą na obwodzie koła). Warto jednak zaznaczyć, że w mojej implementacji udało się uniknąć pełnej degradacji do czasu kwadratowego, zachowując trend $O(n \log n)$ z widocznym, potężnym narzutem czasowym na wywołania rekurencji.
- **Średni przypadek:** $O(n \log n)$ - Dla większości typowych układów danych (losowe, trójkąt, skupisko) algorytm optymalnie dzieli problem na mniejsze części i sprawnie eliminuje punkty leżące wewnątrz obwodu.
- **Najlepszy przypadek:** $O(n)$ - Teoretycznie występuje w sytuacji, gdy już w pierwszych krokach początkowy podział odrzuca niemal wszystkie punkty jako leżące wewnątrz, a drzewo rekurencji jest bardzo płytke.

6. Chan's Algorithm

Algorytm Chana to zaawansowana metoda wyznaczania otoczki wypukłej, która sprytnie łączy w sobie zalety dwóch algorytmów - Łańcucha Monotonicznego (lub Skanowania Grahama) oraz Marszu Jarvisa. Dzięki temu osiąga optymalną złożoność czasową uzależnioną od rozmiaru wyjścia, wynoszącą $O(n \log h)$, gdzie h to ostateczna liczba punktów na gotowej otoczce. Działanie algorytmu opiera się na zgadywaniu rozmiaru otoczki i możemy je podzielić na kilka etapów.

1. Zgadywanie rozmiaru otoczki

Na początku nie wiemy, z ilu punktów będzie składać się ostateczna otoczka. Zakładamy więc bardzo mały rozmiar, zazwyczaj oznaczany jako m (na przykład $m = 4$). Jeśli w trakcie działania algorytmu okaże się, że m jest zbyt małe aby zamknąć wielokąt, przerywamy obliczenia, drastycznie zwiększamy m (najczęściej podnosząc je do kwadratu) i próbujemy od nowa. Takie agresywne potęgowanie gwarantuje, że nie będziemy musieli powtarzać obliczeń zbyt wiele razy, nawet dla bardzo dużych zbiorów.

2. Podział na podzbiory i budowa podotoczek

Dla aktualnie wybranego rozmiaru m , dzielimy cały nasz zbiór n punktów na mniejsze grupy, z których każda liczy maksymalnie m elementów. Następnie dla każdej z tych małych grup wyliczamy jej własną otoczkę wypukłą - tak zwaną podotoczkę. Ponieważ zbiory są małe, wykorzystujemy do tego jako narzędzie pomocnicze inny gotowy algorytm, taki jak Łańcuch Monotoniczny.

3. Zmodyfikowany Marsz Jarvisa

Po przygotowaniu podotoczek, znajdujemy absolutny punkt startowy i rozpoczynamy Marsz Jarvisa. Główna różnica i optymalizacja polega na tym, że szukając kolejnego punktu, nie sprawdzamy już każdego punktu w zbiorze jeden po drugim. Zamiast tego z naszego punktu bieżącego wyznaczamy punkty styczne do każdej z wygenerowanych wcześniej podotoczek. Następnie porównujemy ze sobą tylko te znalezione styczne, wykorzystując do tego znak iloczynu wektorowego. Wybieramy styczną, która tworzy największy skręt w lewo względem punktu bieżącego, a jej koniec staje się kolejnym wierzchołkiem naszej głównej otoczki.

4. Weryfikacja i zamknięcie

Proces szukania stycznych i dodawania nowych wierzchołków powtarzamy maksymalnie m razy. Jeśli przed upływem m kroków nasz nowo dodany wierzchołek okaże się punktem startowym, oznacza to pomyślne zamknięcie obwodu. Zgadliśmy wystarczająco duże m i możemy zwrócić gotową otoczkę. Jeśli jednak zrobimy m kroków, a obwód nadal nie będzie zamknięty, oznacza to, że prawdziwa otoczka ma więcej niż m wierzchołków. Algorytm przerywa wtedy pętlę, wraca do pierwszego etapu, zwiększa m (podnosząc do kwadratu) i ponawia próbę podziału.

Algorytm 5: Algorytm Chana

```
1: procedure FINDTANGENT(hull, p)
2:   best ← hull[0]
3:   i ← 1
4:   n ← ilość hull
5:   while i < n do
6:     cross ← p.cross_product(best, hull[i])
7:     if (cross < -ε) lub (|cross| ≤ ε i p.dist_sq(hull[i]) > p.dist_sq(best)) then
8:       best ← hull[i]
9:     end
10:    i ← i + 1
11:  end
12:  return best
13: end
```

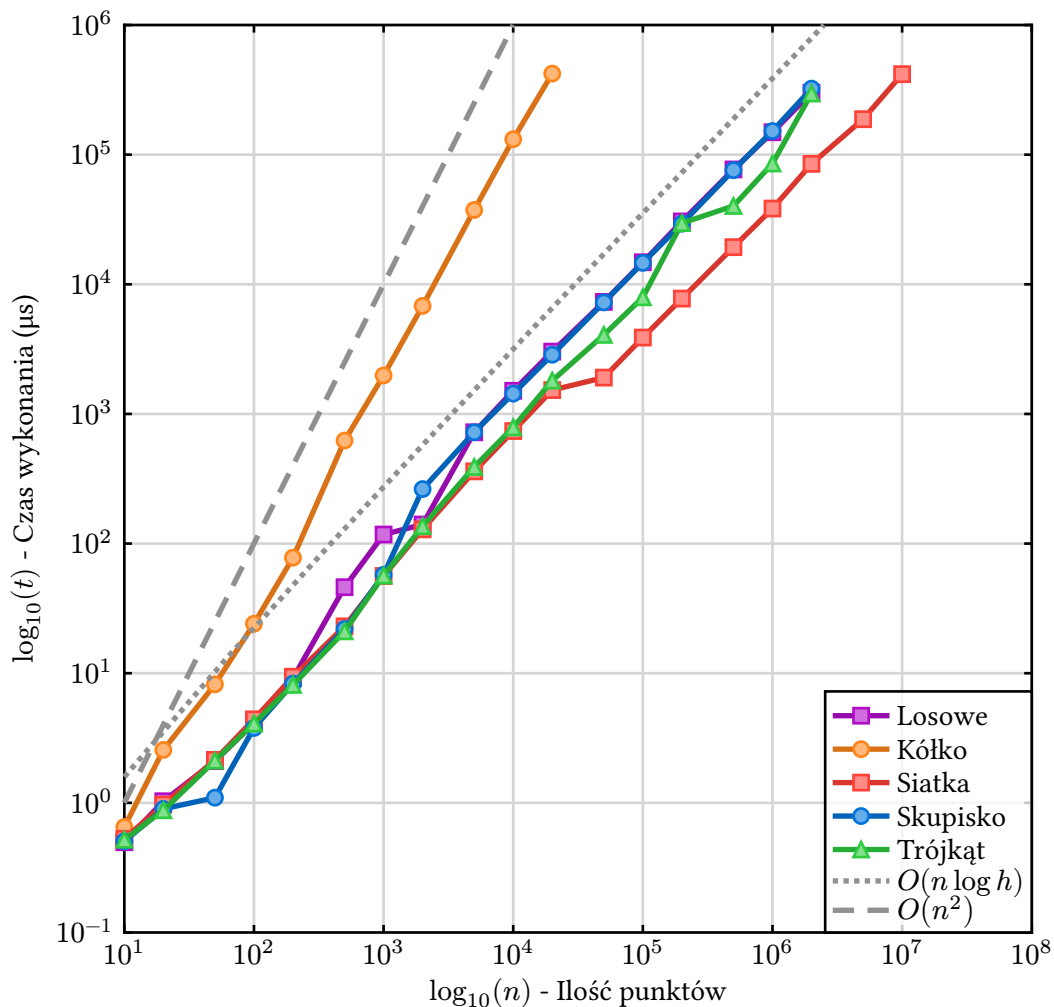
```

14:
15: procedure CHANSALGORITHM(points)
16:    $n \leftarrow$  ilość points
17:   if  $n < 3$  then
18:     return points
19:   end
20:
21:   start  $\leftarrow$  punkt z points o najmniejszym  $Y$ , a w razie remisu o najmniejszym  $X$ 
22:    $t \leftarrow 1$ 
23:
24:   while  $t \leq 30$  do
25:      $m \leftarrow$  minimum z  $2^{2^t}$  oraz  $n$ 
26:
27:     ▷ Budowa podotoczek (Sub-hulls)
28:     hulls  $\leftarrow$  pusta lista
29:      $i \leftarrow 0$ 
30:     while  $i < n$  do
31:       end  $\leftarrow$  minimum z  $i + m$  oraz  $n$ 
32:       group  $\leftarrow$  points[ $i \dots \text{end} - 1$ ]
33:       dodaj graham_scan[group] do hulls
34:        $i \leftarrow i + m$ 
35:     end
36:
37:     ▷ Marsz Jarvisa (wykonujący maksymalnie  $m$  kroków)
38:     hull  $\leftarrow$  pusty stos
39:     dodaj start na stos hull
40:     closed  $\leftarrow$  fałsz
41:     step  $\leftarrow 0$ 
42:     while step  $< m$  do
43:       cur  $\leftarrow$  szczyt stosu hull
44:       next  $\leftarrow$  FINDTANGENT(hulls[0], cur)
45:        $k \leftarrow 1$ 
46:       while  $k < \text{ilość hulls}$  do
47:         cand  $\leftarrow$  FINDTANGENT(hulls[ $k$ ], cur)
48:         cross  $\leftarrow$  cur.cross_product(next, cand)
49:         if cross  $< -\varepsilon$  lub ( $|\text{cross}| \leq \varepsilon$  i cur.dist_sq(cand)  $>$  cur.dist_sq(next)) then
50:           next  $\leftarrow$  cand
51:         end
52:          $k \leftarrow k + 1$ 
53:       end
54:
55:       if next = start then
56:         closed  $\leftarrow$  prawda
57:         BREAK
58:       end
59:       dodaj next na stos hull
60:       step  $\leftarrow$  step + 1
61:     end
62:
63:     if closed then
64:       return hull
65:     end
66:
67:      $t \leftarrow t + 1$ 
68:   end
69: end

```

Hipotezy badawcze dla poszczególnych scenariuszy

1. **Punkty losowe:** Algorytm powinien radzić sobie dobrze - mała liczba punktów na otoczce względem n sprawia, że oczekuję wykresu zbliżonego do $O(n \log n)$, podobnego do Quick Hull.
2. **Punkty na obwodzie koła:** worst-case - Wszystkie punkty leżą na otoczce, więc algorytm będzie wielokrotnie powtarzał fazę z podwajaniem m , zanim trafi na właściwe $h = n$. Spodziewam się najdłuższego czasu wykonania bliskiego $O(n^2)$.
3. **Punkty na siatce:** best-case - h zmierza do stałej, więc algorytm bardzo szybko trafi na właściwe m i nie będzie musiał powtarzać faz. Oczekuję wykresu niemal liniowego.
4. **Punkty w skupisku:** Otoczkę wyznacza nieliczne zewnętrzne koło, h jest małe względem n . Faza z podwajaniem zakończy się szybko, oczekuję czasu zbliżonego do danych losowych.
5. **Punkty w trójkącie:** h pozostaje małe i w przybliżeniu stałe, więc algorytm powinien zachowywać się podobnie do scenariusza z siatką. Skupisko przy krawędzi AB nie powinno znacząco wpłynąć na liczbę iteracji faz.



Rysunek 5: Czas wykonania Chan's Algorithm - porównanie scenariuszy danych wejściowych z liniami referencyjnymi $O(n \log h)$ i $O(n^2)$, gdzie $h = \sqrt{n}$

Analizując Rysunek 5, można zauważyć, jak różnie zachowuje się algorytm Chana w zależności od ułożenia punktów, co w pełni potwierdza postawione wcześniej hipotezy. Zdecydowanie najgorzej wypada scenariusz z punktami na obwodzie koła. Linia dla tego przypadku szybko pnie się w górę, uciekając od reszty i pokrywając się z przerywaną linią dla czasu $O(n^2)$. Zgadza się to z założeniem, że dla takich danych algorytm musi wielokrotnie powtarzać fazę podwajania rozmiaru grup, co w praktyce ogromnie go spowalnia. Z drugiej strony, najlepiej wypadły punkty na siatce. Wykres jest

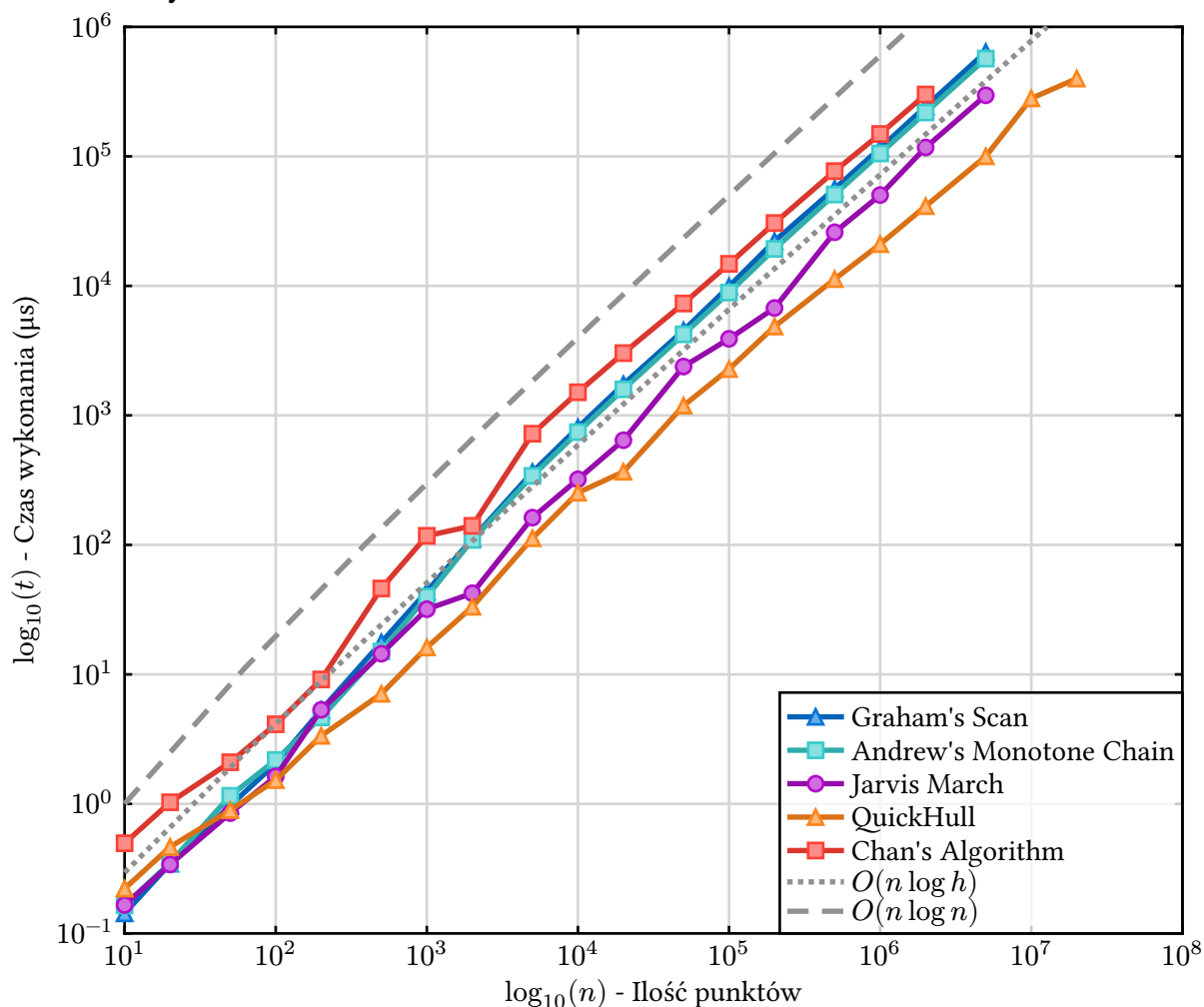
tam bardzo niski i płaski, ponieważ punktów tworzących powłokę jest bardzo mało, przez co algorytm szybko odgaduje odpowiedni limit i kończy pracę. Pozostałe trzy scenariusze, czyli punkty losowe, trójkąt oraz skupisko, zachowują się niemal identycznie i tworzą jedną zwartą wiązkę pośrodku wykresu, rosnącą w oczekiwanym tempie $O(n \log h)$. Dla punktów losowych i trójkąta algorytm działa sprawnie dzięki ogólnie małej liczbie punktów na powłoce. Co ciekawe, w przypadku skupiska mała liczba punktów na tym wielkim, zewnętrznym okręgu sprawiła, że algorytm zupełnie zignorował gęsto upakowany środek i poradził sobie z tym zadaniem tak samo szybko, jak ze zwykłymi danymi losowymi. Wszystkie początkowe założenia okazały się więc trafne.

Czasowa złożoność obliczeniowa

- **Najgorszy przypadek:** $O(n^2)$ - Teoretyczna złożoność Chana wynosi $O(n \log h)$, co dla $h = n$ powinno dać górne ograniczenie $O(n \log n)$. Jednak wykres wyraźnie pokazuje degradację do $O(n^2)$. Wynika to z technicznego narzutu na wielokrotne niszczenie i budowanie nowych struktur w fazie podwajania parametru m , połączonego z pesymistycznym przypadkiem kroku skalania (zbyt kosztowny marsz Jarvisa po niepotrzebnie dublowanych podpowłokach).
- **Średni przypadek:** $O(n \log h)$ - Algorytm skaluje się proporcjonalnie do ilości punktów w zbiorze oraz logarytmu z liczby punktów na ostatecznej powłoce. Widoczne jest to w głównej wiązce na wykresie.
- **Najlepszy przypadek:** $O(n)$ - Kiedy liczba punktów na otoczce jest stała i bardzo mała. Algorytm wykonuje tylko początkowe iteracje (dla minimalnych wartości m) i od razu znajduje otoczkę, dzięki czemu omija potężne narzuty z fazy wykładniczego skalowania m .

7. Porównanie algorytmów

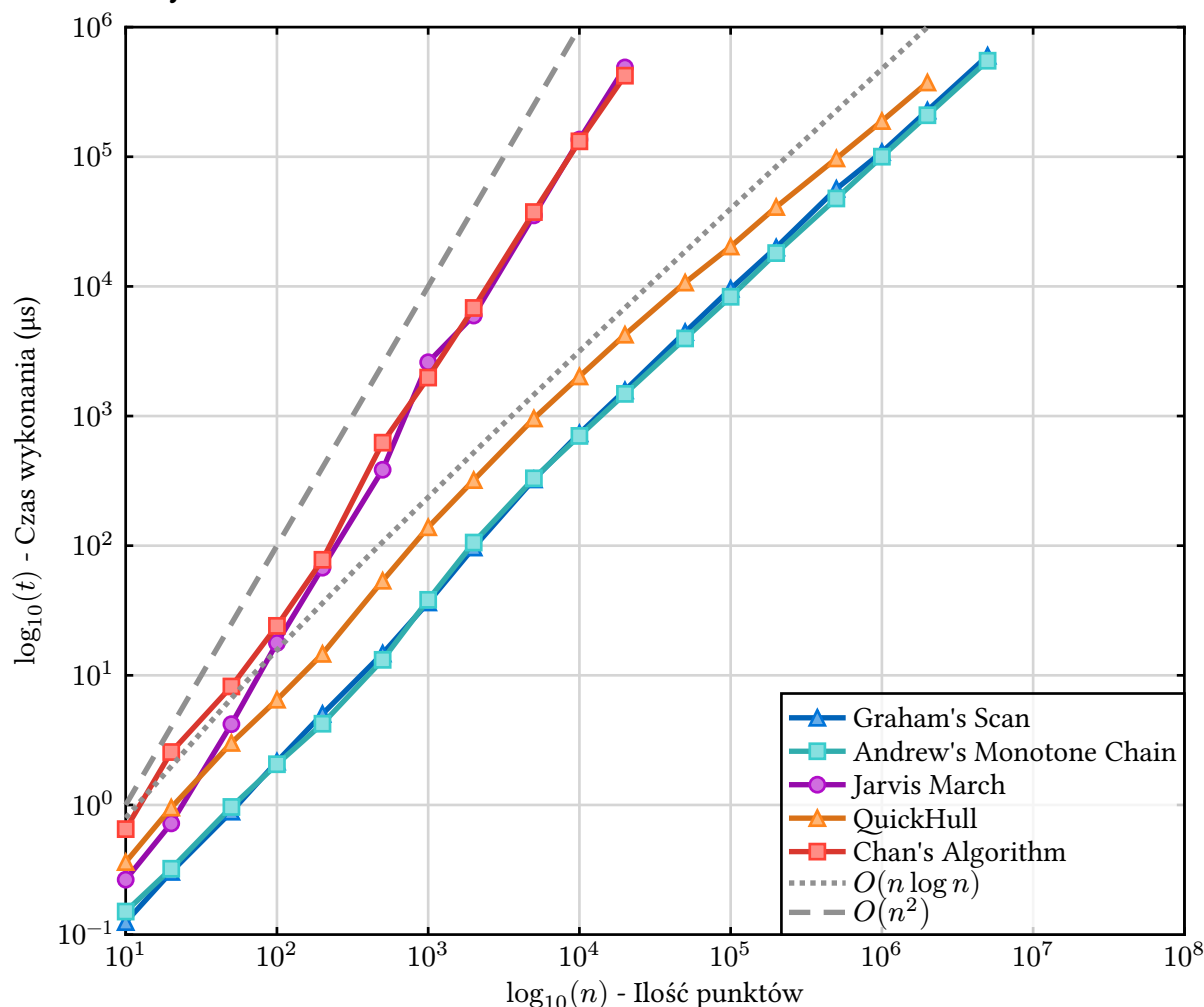
7.1. Punkty losowe



Rysunek 6: Porównanie algorytmów - punkty na siatce z liniami referencyjnymi $O(n \log h)$ i $O(n \log n)$, gdzie $h = \sqrt{n}$

Na Rysunek 6 dla punktów losowych, bezkonkurencyjnym zwycięzcą okazuje się QuickHull. Jego linia znajduje się najniżej na wykresie, co oznacza, że działa on zdecydowanie najszybciej dla tego zestawu danych. Zaraz za nim jest Jarvis March, co wynika z tego, że dla losowo rozrzuconych punktów otoczka wypukła jest stosunkowo mała w stosunku do wszystkich punktów, co ten algorytm potrafi dobrze wykorzystać. Algorytmy Grahama i Andrewsa zachowują się praktycznie tak samo - ich linie nakładają się na siebie. Działają one zauważalnie wolniej od liderów, ponieważ zawsze muszą posortować wszystkie punkty na samym początku. Najgorzej w tym zestawieniu wypada algorytm Chana. Mimo dobrych założeń teoretycznych, widać, że w praktyce jego implementacja narzuca za duży overhead, przez co jest najwolniejszy ze wszystkich testowanych rozwiązań dla losowych danych.

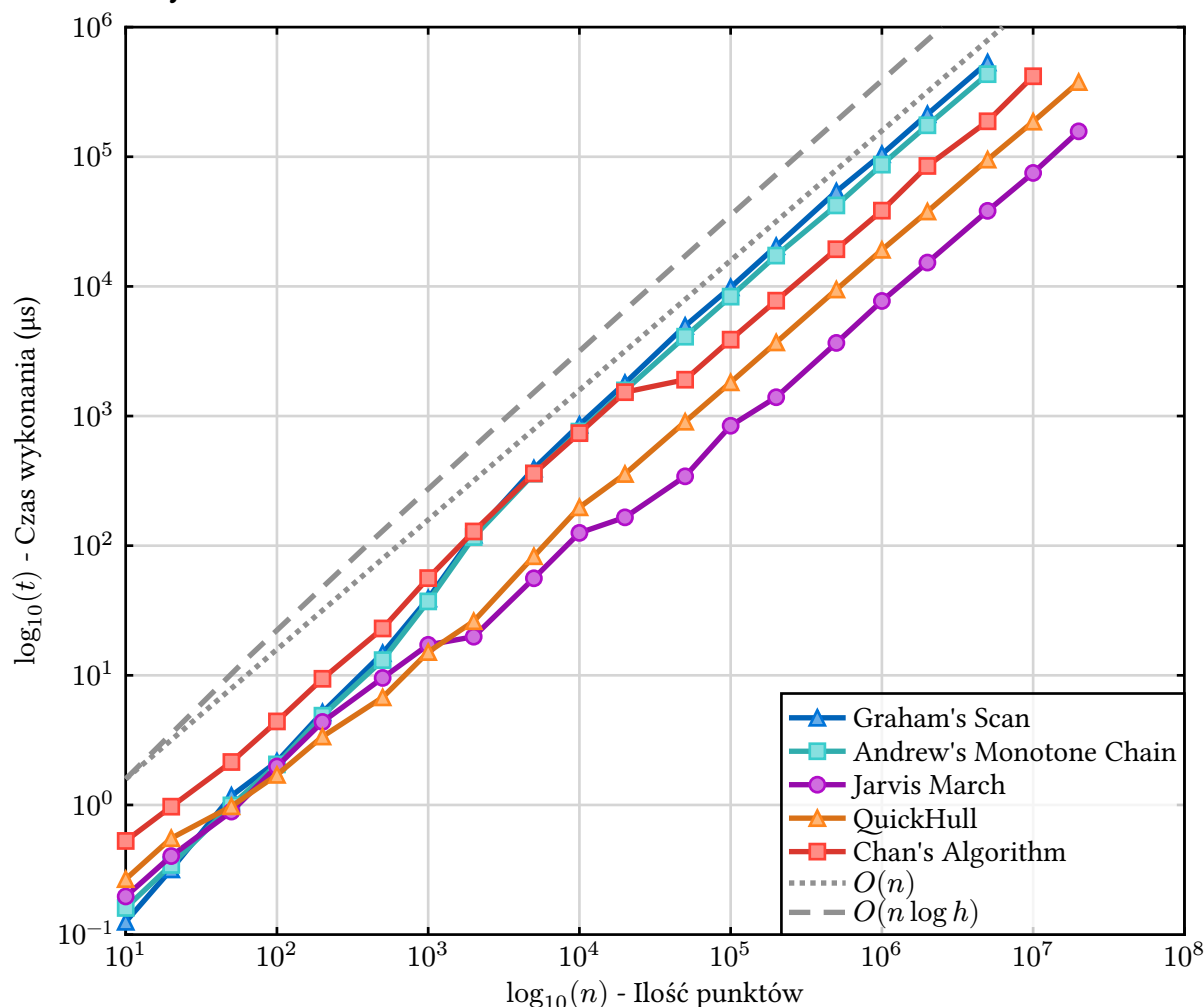
7.2. Punkty na obwodzie koła



Rysunek 7: Porównanie algorytmów - punkty na obwodzie koła z liniami referencyjnymi $O(n \log h)$ i $O(n^2)$, gdzie $h = \sqrt{n}$

W przypadku punktów ułożonych na obwodzie koła widać wyraźny podział algorytmów na dwie skrajne grupy. Zdecydowanymi zwycięzcami w tym scenariuszu są Graham's Scan i Andrew's Monotone Chain. Ich linie praktycznie całkowicie się nakładają na samym dole wykresu. Wynika to z tego, że dla tych algorytmów fakt leżenia wszystkich punktów na otoczce nie stanowi żadnego problemu – ich czas działania i tak opiera się głównie na początkowym sortowaniu. Nieco wyżej znajduje się QuickHull. Mimo że jego podział rekurencyjny nie odrzuca tutaj żadnych punktów wewnątrz obwodu, to algorytm unika całkowitej degradacji do czasu kwadratowego i nadal rośnie w tempie $O(n \log n)$, choć widać po nim wyraźny narzut obliczeniowy w porównaniu do liderów. Z kolei Jarvis March i algorytm Chana radzą sobie tutaj fatalnie, a ich wykresy stromo pną się w górę podążając za linią referencyjną $O(n^2)$. Dla algorytmu Jarvisa jest to przewidywalne zachowanie, ponieważ otoczka składa się ze wszystkich podanych punktów ($h = n$), co wymusza iterowanie po całym zbiorze w każdym możliwym kroku. Algorytm Chana również degraduje tutaj do czasu kwadratowego z powodu ogromnego narzutu na wielokrotne, bezsensowne powtarzanie faz łączenia przy tak wielkim parametrze h .

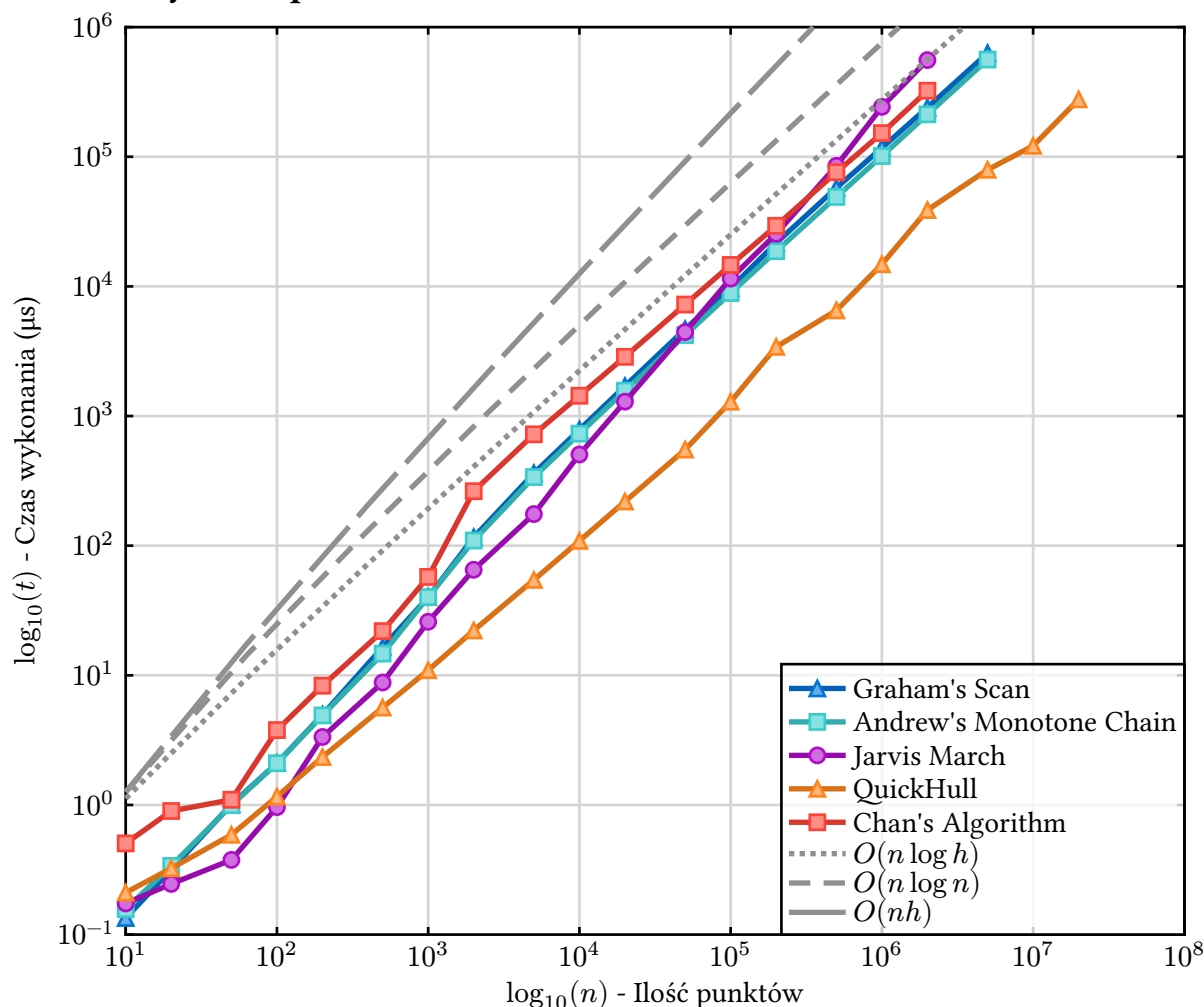
7.3. Punkty na siatce



Rysunek 8: Porównanie algorytmów - punkty na siatce z liniami referencyjnymi $O(n)$ i $O(n \log h)$, gdzie $h = \sqrt{n}$

W przypadku punktów rozmieszczonych na regularnej siatce zwycięzcą okazuje się Jarvis March. Jego linia leży najniżej na wykresie, rosnąc liniowo. Dzieje się tak, ponieważ otoczkę wypukłą dla takiego układu tworzą zaledwie cztery skrajne punkty (rogi siatki). Jarvis March świetnie to wykorzystuje, wykonując bardzo mało kroków. Zaraz za nim jest QuickHull, który również bardzo sprawnie odrzuca większość punktów leżących wewnątrz siatki już na wczesnych etapach działania. Algorytm Chana wypada tutaj średnio. Choć zyskuje na małej liczbie punktów na otoczce, to widać, że jego skomplikowana struktura i początkowy narzut sprawiają, że działa wolniej od Jarvisa, szczególnie dla mniejszych zbiorów danych. Najgorzej na dużych zbiorach radzą sobie Graham's Scan i Andrew's Monotone Chain, których wykresy tradycyjnie pokrywają się na samej górze. Wynika to z faktu, że niezależnie od tego, jak mało punktów ostatecznie trafia na powłokę, oba te algorytmy muszą na samym początku bezwzględnie posortować całą pulę danych. Faza sortowania narzuca czas $O(n \log n)$, co w tym wyjątkowo sprzyjającym scenariuszu staje się głównym wąskim gardłem i sprawia, że wypadają one najslabiej.

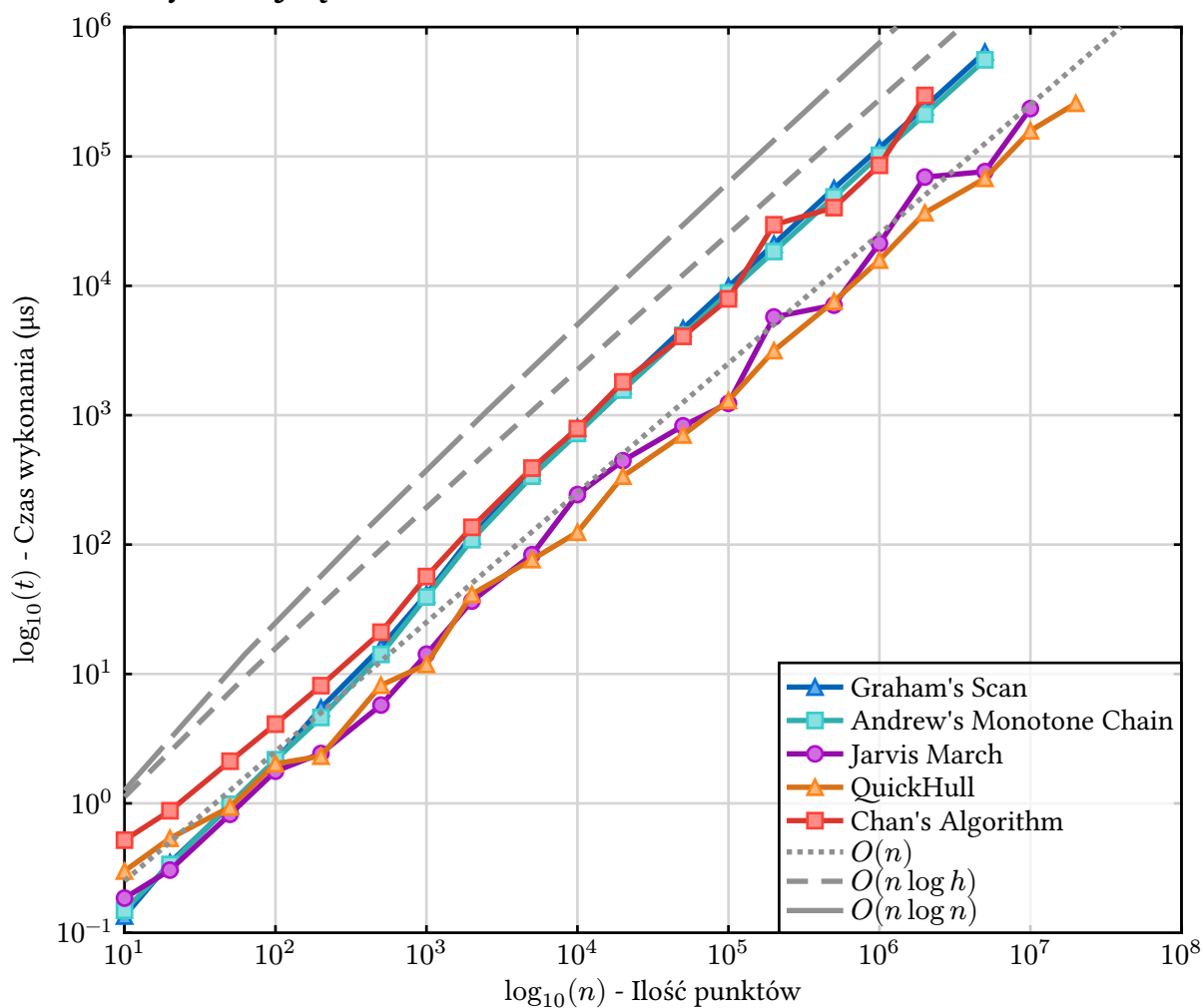
7.4. Punkty w skupisku



Rysunek 9: Porównanie algorytmów - punkty w skupisku z liniami referencyjnymi $O(n \log h)$, $O(n \log n)$ i $O(nh)$, gdzie $h = \sqrt{n}$

W scenariuszu ze skupiskiem punktów bezkonkurencyjny okazuje się QuickHull. Wykres pokazuje nam, że ten algorytm rewelacyjnie radzi sobie z masowym i szybkim odrzucaniem gęsto upakowanych punktów leżących wewnątrz otoczki. Środek zajmują Graham's Scan i Andrew's Monotone Chain, których wykresy tradycyjnie już całkowicie się pokrywają. Oba rosną stabilnie w tempie $O(n \log n)$. Dla nich gęste skupisko w środku nie stanowi ani ułatwienia, ani utrudnienia, ponieważ i tak główny czas tracą na obowiązkowe posortowanie wszystkich punktów na samym starcie. Algorytm Chana przez większość czasu jest po prostu wolniejszy od tej dwójki, co ponownie potwierdza spory narzut obliczeniowy wynikający z jego skomplikowanej mechaniki i łączenia podzbiorów. Najciekawiej jednak zachowuje się tutaj Jarvis March. Choć dla bardzo małej liczby punktów jest najszybszy, to jego wykres rośnie najstromej. Przy dużych zbiorach danych przebija wszystkie inne linie i ostatecznie staje się najwolniejszym algorytmem w całym zestawieniu. Dzieje się tak, ponieważ 10% punktów zostało rozrzuconych na wielkim, zewnętrznym okręgu, co bardzo mocno podbiło wartość parametru h . Ponieważ Jarvis musi przeszukiwać całą pulę punktów przy każdym kolejnym wierzchołku, tak duża powłoka wypukła drastycznie zwiększyła jego czas wykonania.

7.5. Punkty w trójkącie



Rysunek 10: Porównanie algorytmów - punkty w trójkącie z liniami referencyjnymi $O(n)$, $O(n \log h)$ i $O(n \log n)$, gdzie $h = \sqrt{n}$

W scenariuszu z punktami rozmieszczonymi wewnątrz trójkąta najlepiej radzą sobie QuickHull oraz Jarvis March, których linie znajdują się na samym dole wykresu. Jarvis March wypada tutaj świetnie, ponieważ powłoka wypukła dla takiego układu składa się zaledwie z kilku punktów. Przy największych zbiorach danych to jednak QuickHull ostatecznie przejmuje prowadzenie i jest najszybszy, bo niezwykle skutecznie i masowo odrzuca zbędne punkty leżące wewnątrz trójkąta. Najślabiej w tym zestawieniu wypadają Graham's Scan, Andrew's Monotone Chain i algorytm Chana. Ich wykresy całkowicie się pokrywają i biegną najwyżej.

8. Wnioski końcowe

Podsumowując wszystkie testy, wyraźnie widać, że wybór odpowiedniego algorytmu wyznaczania otoczki wypukłej mocno zależy od ułożenia danych wejściowych. Nie ma jednego rozwiązania, które wygrywa w każdej sytuacji.

W większości typowych scenariuszy (takich jak punkty losowe czy w skupisku) bezkonkurencyjny okazuje się QuickHull, który potrafi bardzo szybko odrzucać punkty leżące wewnątrz otoczki. Jarvis March to algorytm skrajności - jest niesamowicie szybki na siatce, gdzie otoczka ma mało wierzchołków, ale drastycznie zwalnia, gdy powłoka jest duża, tak jak na obwodzie koła lub gdy wiele punktów wyląduje na zewnętrznym okręgu. Z kolei Skanowanie Grahama i Łańcuch Monotoniczny to bardzo stabilne, przewidywalne algorytmy, które niezależnie od rozkładu punktów zawsze działają w czasie zdominowanym przez początkowe sortowanie.

Największym rozczarowaniem całego zestawienia okazał się algorytm Chana. Teoretycznie miał on łączyć najlepsze cechy innych metod i dawać optymalny czas zależny od wielkości wyjścia. W praktyce jednak wypadł on słabo w niemal każdym scenariuszu. Dlaczego tak się dzieje? Wynika to z tego, że implementacja algorytmu Chana jest bardzo skomplikowana i narzuca potężny overhead. Program traci mnóstwo czasu na wielokrotne niszczenie i budowanie nowych struktur w fazie potęgowania i zwiększania parametru m . Pokazuje nam to, że świetna złożoność w teorii to jedno, a praktyczne spowolnienia wynikające ze skomplikowanej mechaniki to coś zupełnie innego.